

Transformer architectures for text and medical records

Matt Engelhard

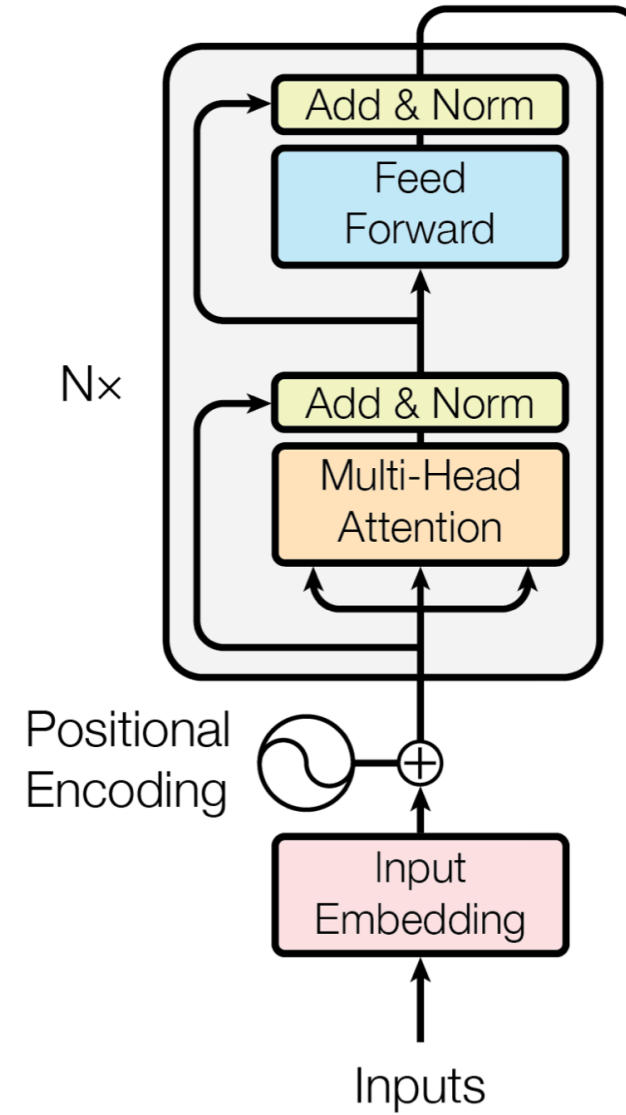
Recall: Transformer Encoder

Input embeddings and positional encodings

- Input embeddings: map text to a sequence of dense vector representations
- Positional encodings: add information about relative position within the sequence (we will return to this)

Transformer encoder stack (N=6 in Vaswani; N=24 in BERT-large; N=96 in GPT-3)

- Multi-head attention: refine representations of words (i.e. the meaning of each token) based on other words
[information exchange across words]
- Add & norm: residual connections (**add**) avoid vanishing gradients in earlier layers; normalizing improves numeric stability
- Feed forward: nonlinear refinement of representations
[information exchange across embedding/representation dimensions]



Recall: Transformer Encoder

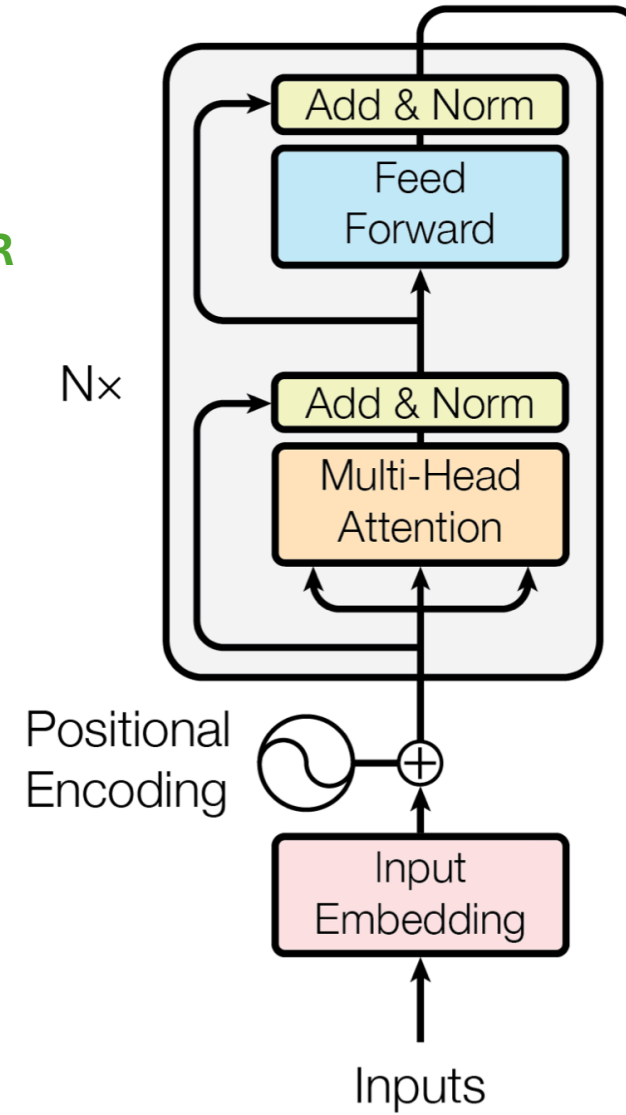
Input embeddings and positional encodings

EMBEDDING LAYER

- Input embeddings: map text to a sequence of dense vector representations
- Positional encodings: add information about relative position within the sequence (we will return to this)

Transformer encoder stack (N=6 in Vaswani; N=24 in BERT-large; N=96 in GPT-3)

- Multi-head attention: refine representations of words (i.e. the meaning of each token) based on other words
[information exchange across words]
- Add & norm: residual connections (**add**) avoid vanishing gradients in earlier layers; normalizing improves numeric stability
- Feed forward: nonlinear refinement of representations
[information exchange across embedding/representation dimensions]



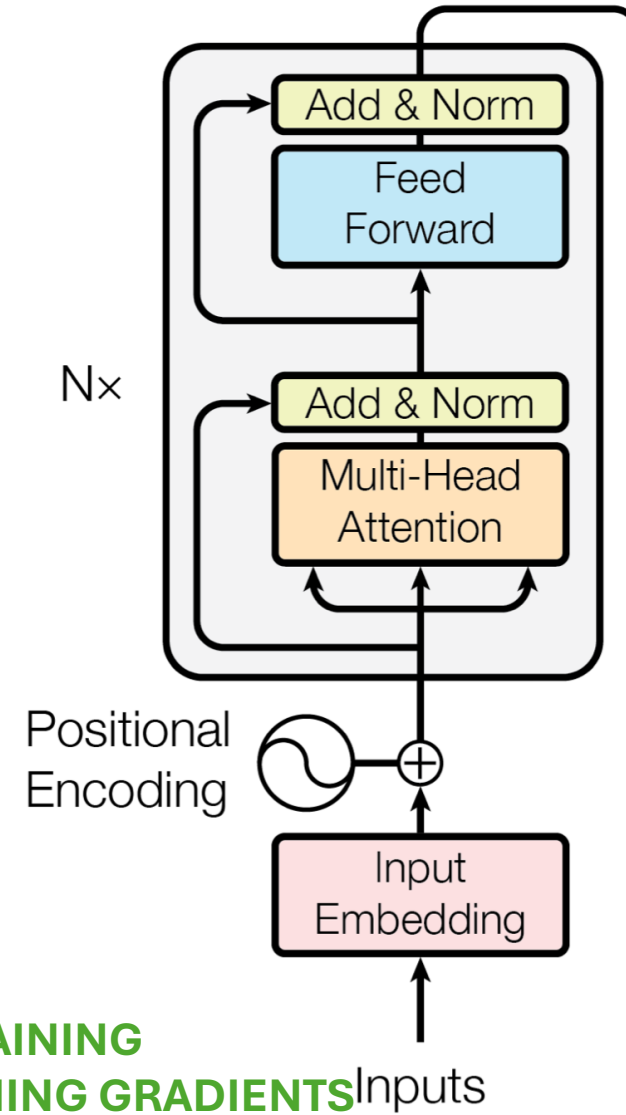
Recall: Transformer Encoder

Input embeddings and positional encodings

- Input embeddings: map text to a sequence of dense vector representations
- Positional encodings: add information about relative position within the sequence (we will return to this)

Transformer encoder stack (N=6 in Vaswani; N=24 in BERT-large; N=96 in GPT-3)

- Multi-head attention: refine representations of words (i.e. the meaning of each token) based on other words
[information exchange across words]
- Add & norm: residual connections (**add**) avoid vanishing gradients in earlier layers; normalizing improves numeric stability
- Feed forward: nonlinear refinement of representations
[information exchange across embedding/representation dimensions]



STABLE TRAINING
NO VANISHING GRADIENTS

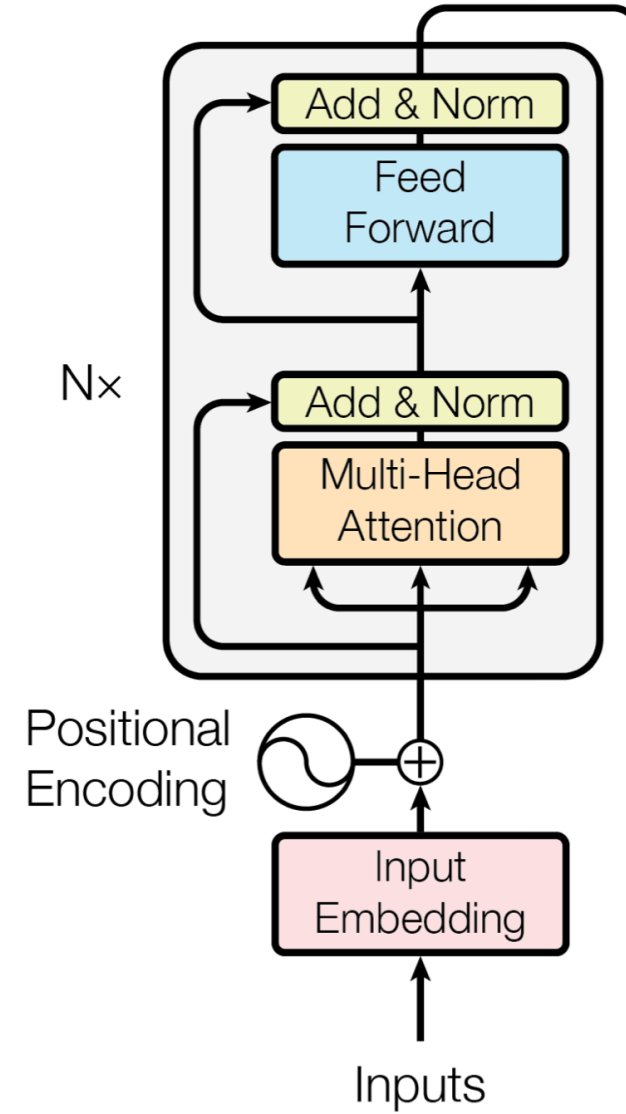
Recall: Transformer Encoder

Input embeddings and positional encodings

- Input embeddings: map text to a sequence of dense vector representations
- Positional encodings: add information about relative position within the sequence (we will return to this)

Transformer encoder stack (N=6 in Vaswani; N=24 in BERT-large; N=96 in GPT-3)

- Multi-head attention: refine representations of words (i.e. the meaning of each token) based on other words
[information exchange across words]
- Add & norm: residual connections (**add**) avoid vanishing gradients in earlier layers; normalizing improves numeric stability
- Feed forward: nonlinear refinement of representations
[information exchange across embedding/representation dimensions]



MLP APPLIED IN PARALLEL

Recall: Transformer Encoder

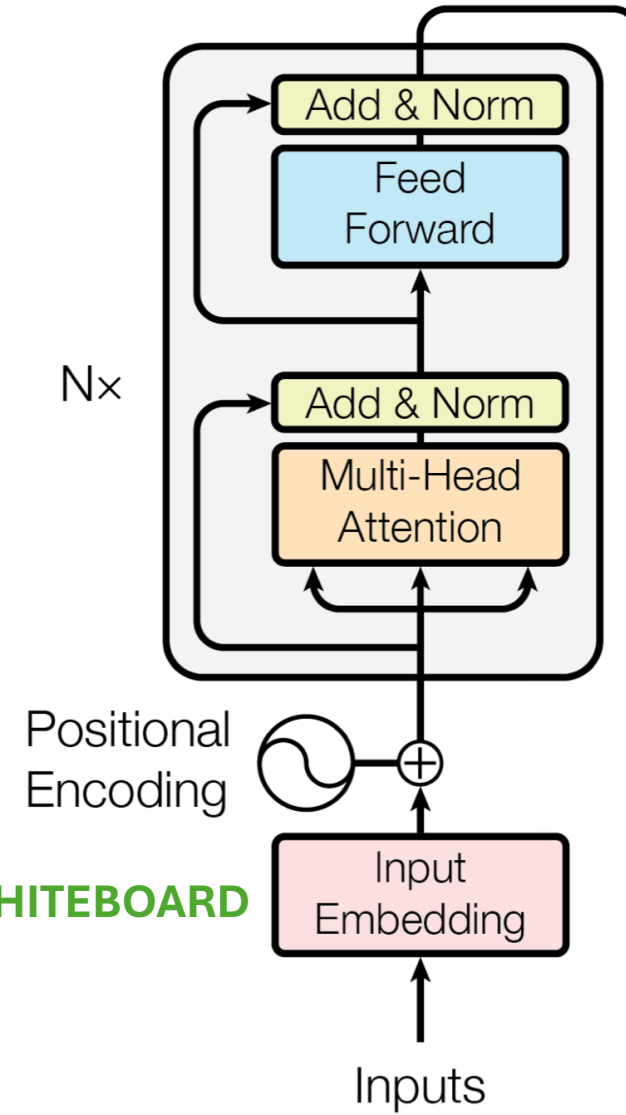
Input embeddings and positional encodings

- Input embeddings: map text to a sequence of dense vector representations
- Positional encodings: add information about relative position within the sequence (we will return to this)

Transformer encoder stack (N=6 in Vaswani; N=24 in BERT-large; N=96 in GPT-3)

- Multi-head attention: refine representations of words (i.e. the meaning of each token) based on other words
[information exchange across words]
- Add & norm: residual connections (**add**) avoid vanishing gradients in earlier layers; normalizing improves numeric stability
- Feed forward: nonlinear refinement of representations
[information exchange across embedding/representation dimensions]

ON THE WHITEBOARD



Recall: Transformer Encoder

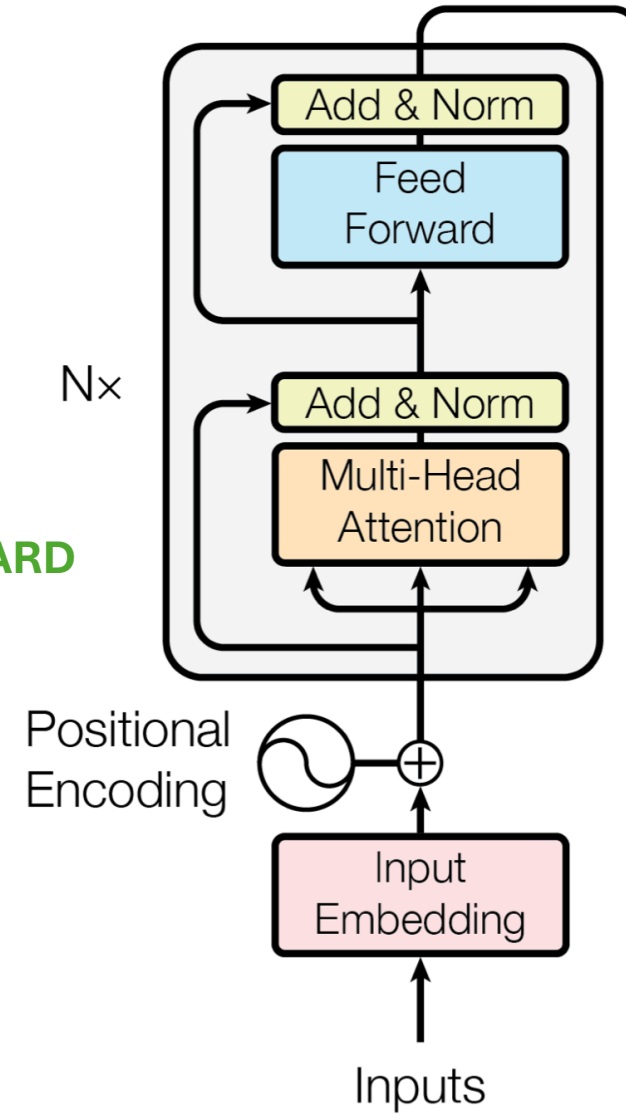
Input embeddings and positional encodings

- Input embeddings: map text to a sequence of dense vector representations
- Positional encodings: add information about relative position within the sequence (we will return to this)

ON THE WHITEBOARD

Transformer encoder stack (N=6 in Vaswani; N=24 in BERT-large; N=96 in GPT-3)

- Multi-head attention: refine representations of words (i.e. the meaning of each token) based on other words
[information exchange across words]
- Add & norm: residual connections (**add**) avoid vanishing gradients in earlier layers; normalizing improves numeric stability
- Feed forward: nonlinear refinement of representations
[information exchange across embedding/representation dimensions]



Training process

Encoder-only models

Training encoder-only architectures:

Masked language modeling (MLM)

Output
representations

x_1

x_2

x_3

x_4

x_5

x_6

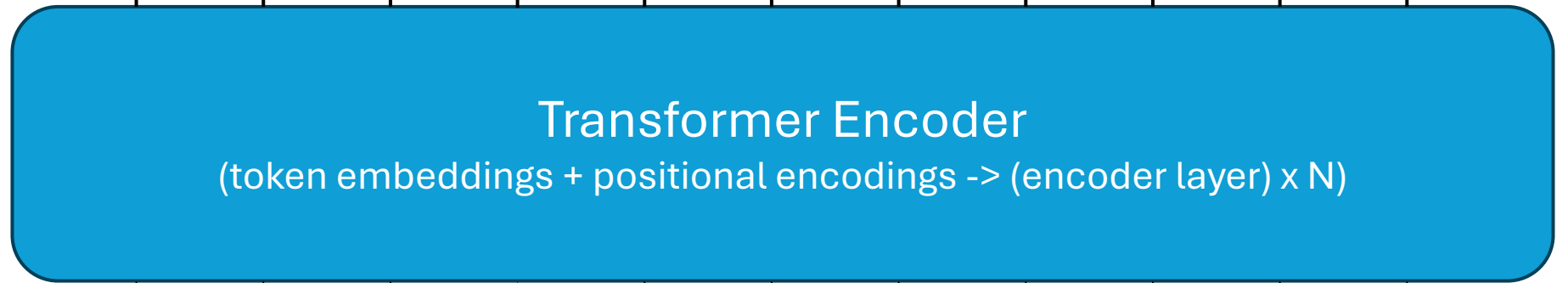
x_7

x_8

x_9

x_{10}

x_{11}



Input tokens

Even

the

smallest

person

can

change

the

course

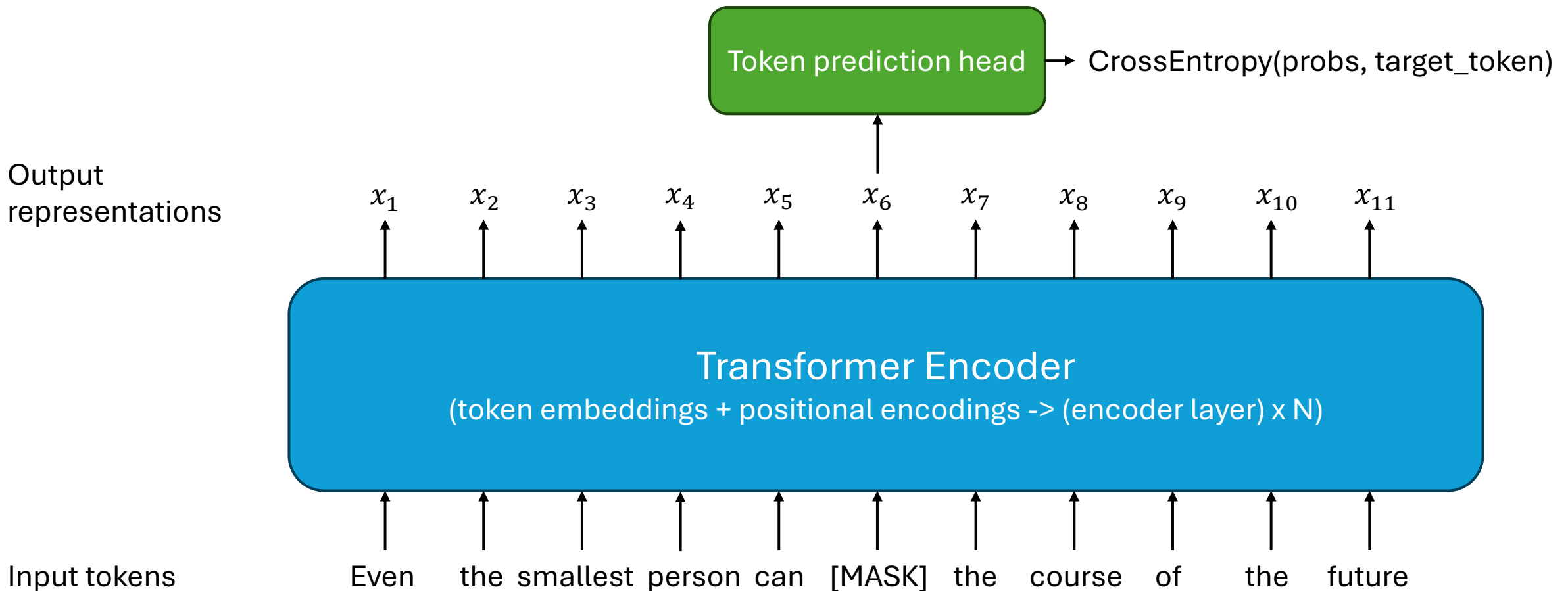
of

the

future

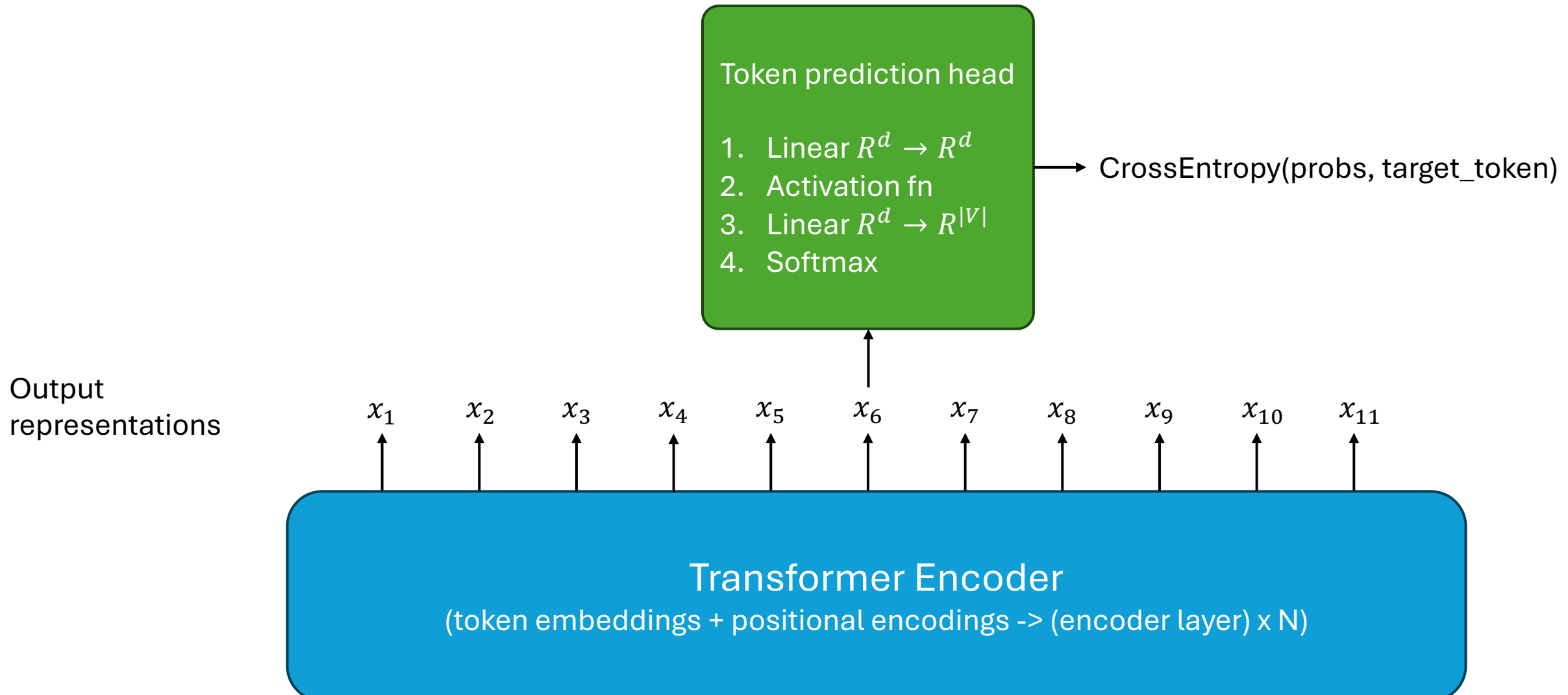
Training encoder-only architectures:

Masked language modeling (MLM)

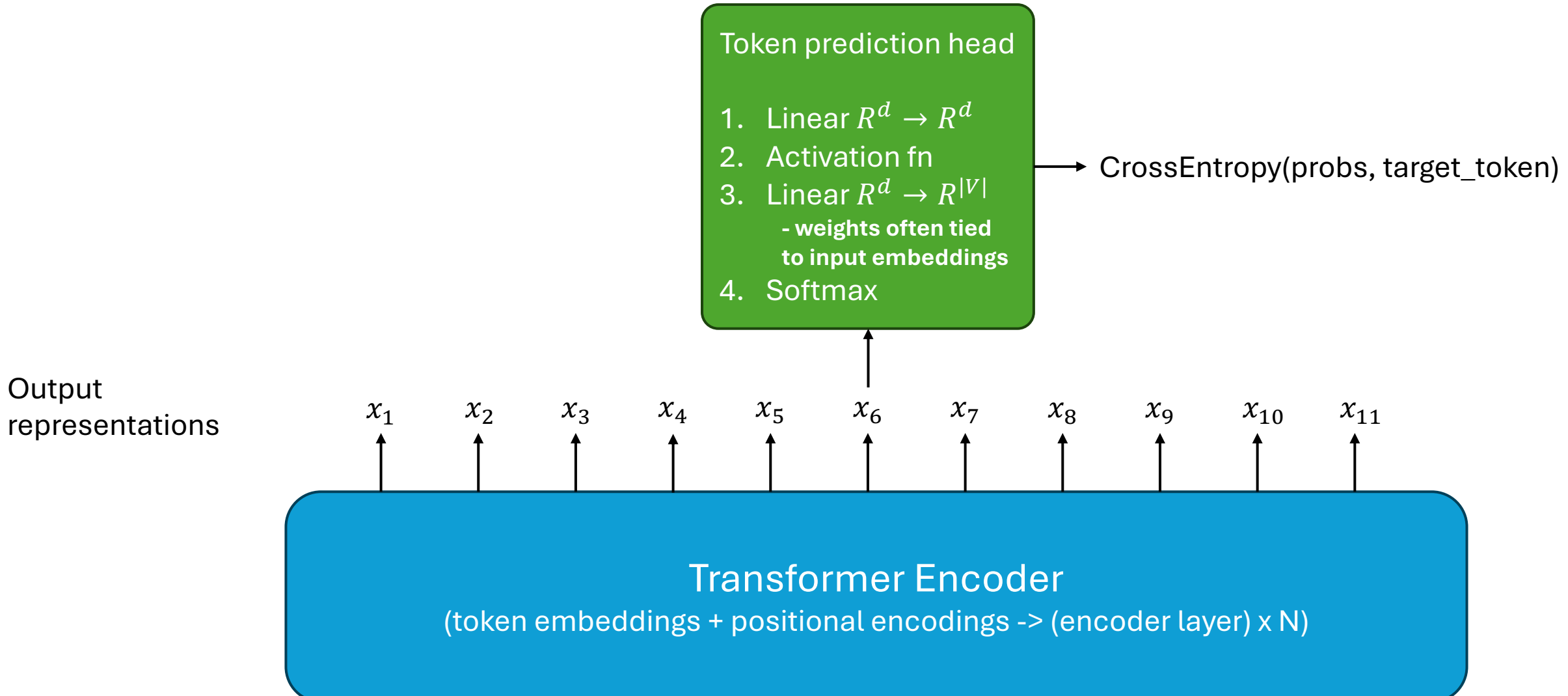


Training encoder-only architectures:

Masked language modeling (MLM)



Training encoder-only architectures: Masked language modeling (MLM)



Special tokens

- [CLS] is a “classification” token prepended to the input sequence
 - final hidden state at the [CLS] position can attend to all other tokens
 - for sentence-level classification tasks, a classification head is added on top of the [CLS] hidden state during fine-tuning
- [MASK] is used for masked language modeling
- [SEP] can be used to separate distinct pieces of the input (e.g. sentences)
- [PAD] is used to pad shorter sequences up to a common length
- [UNK] is used for tokens not in the vocabulary

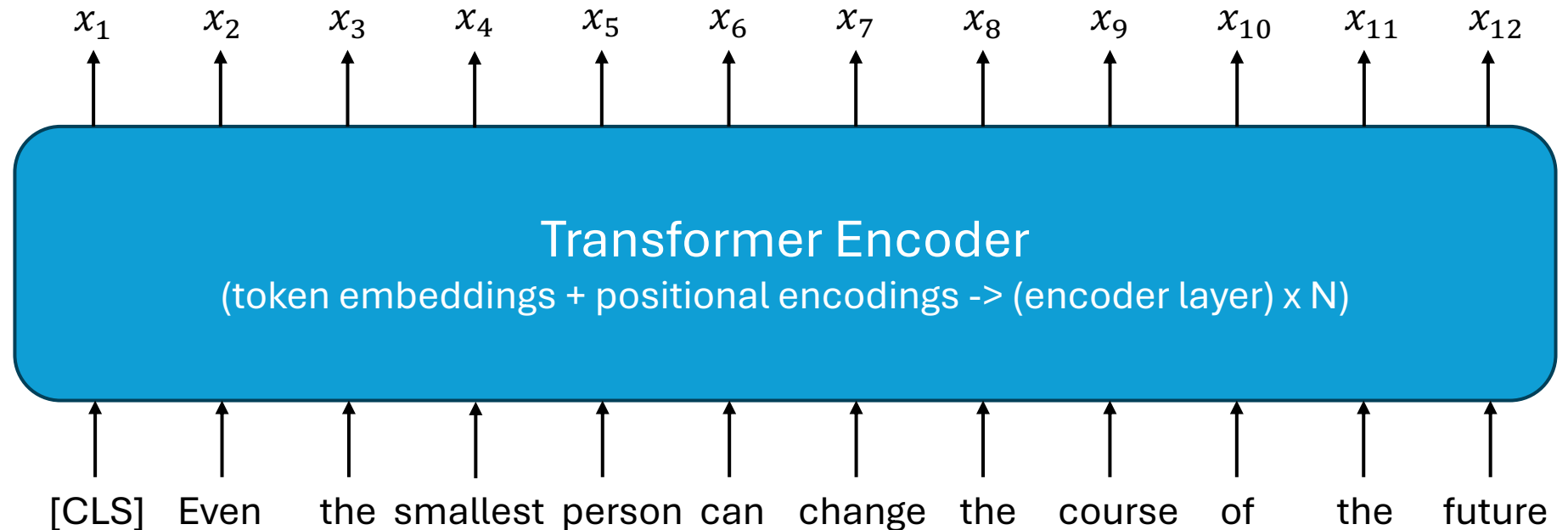
How do we use the output?

Encoder-only models

Encoder gives a sequence of representations

Consider: How are these different / better than Word2Vec embeddings?

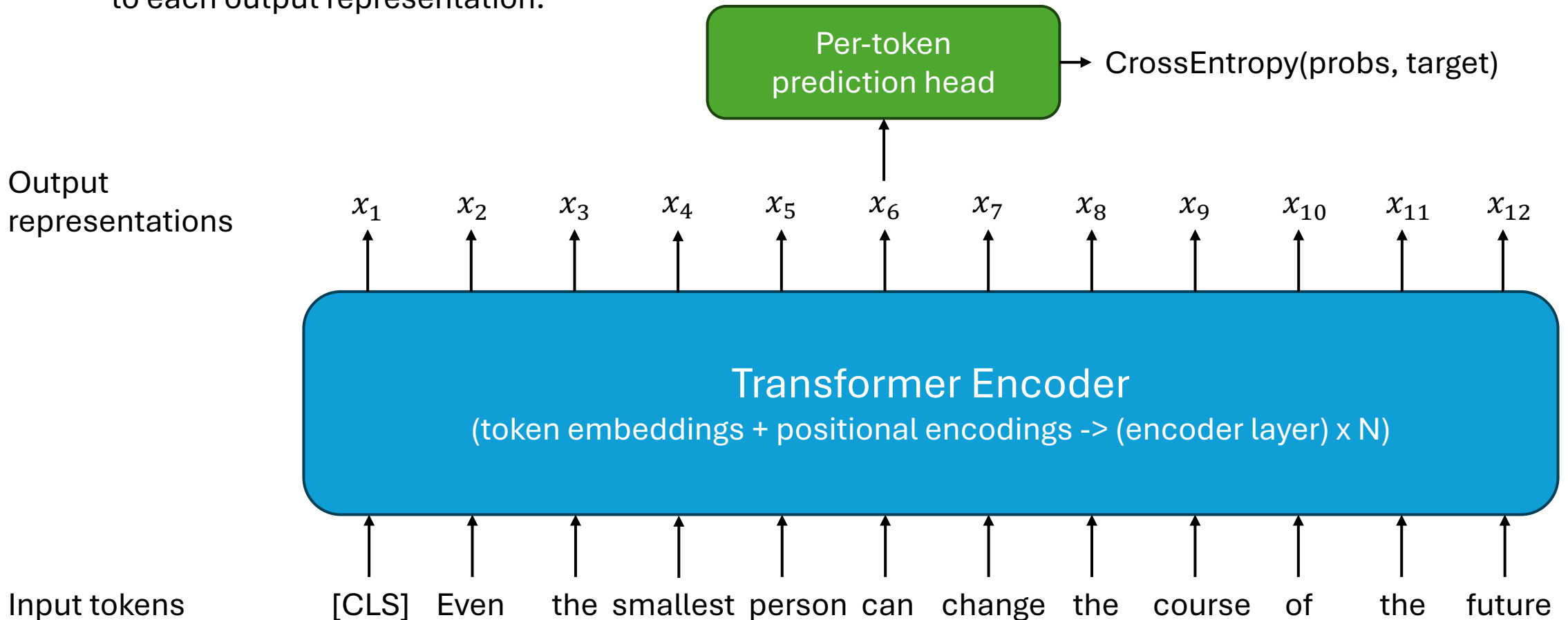
Output
representations



Input tokens

Encoder gives a sequence of representations

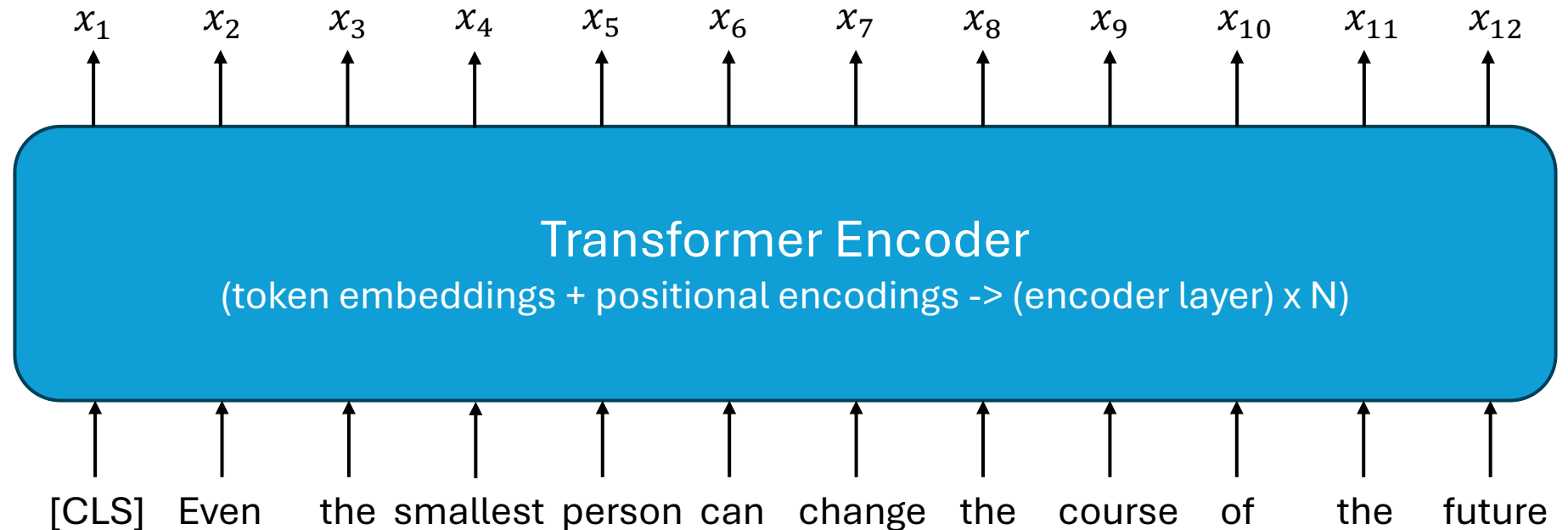
Using this for token-level prediction is straightforward: apply prediction head to each output representation.



Options for document-level prediction

Consider: How do we make document-level predictions?

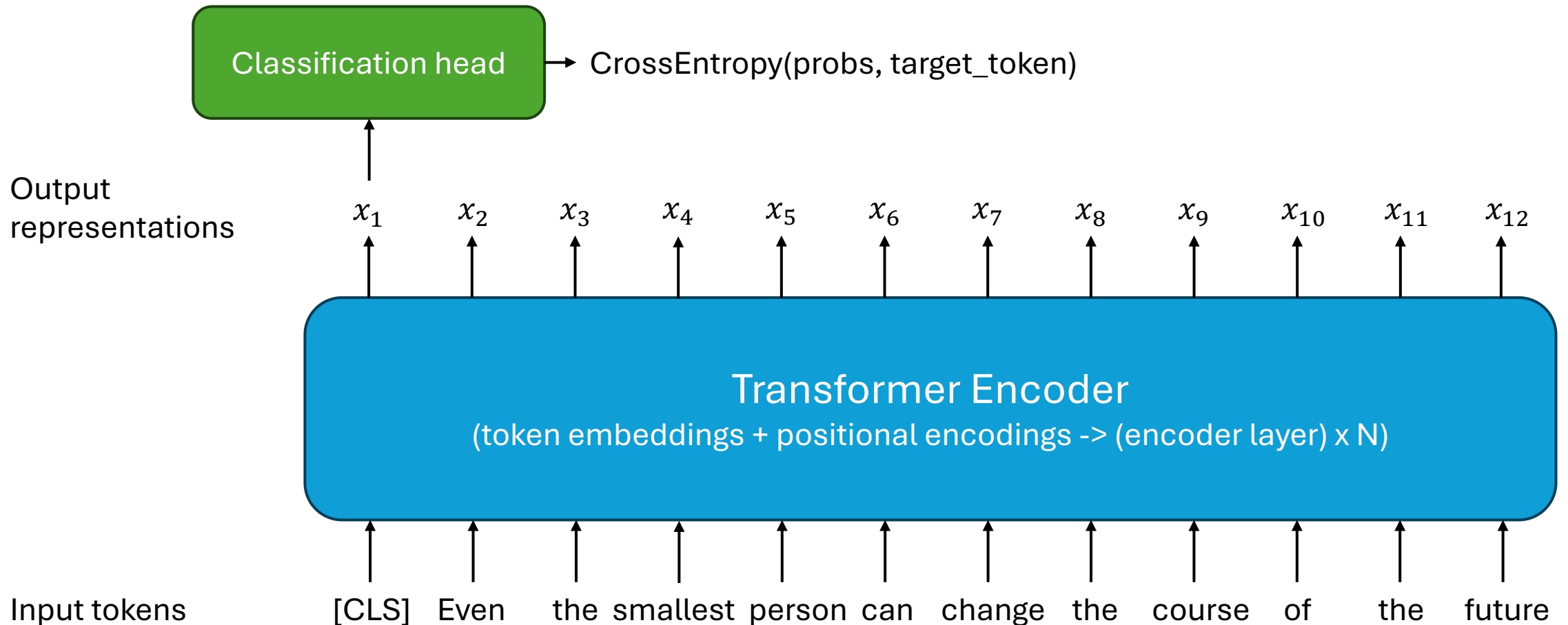
Output
representations



Input tokens

Options for sentence-level prediction

Most common approach: predict from [CLS] token representation

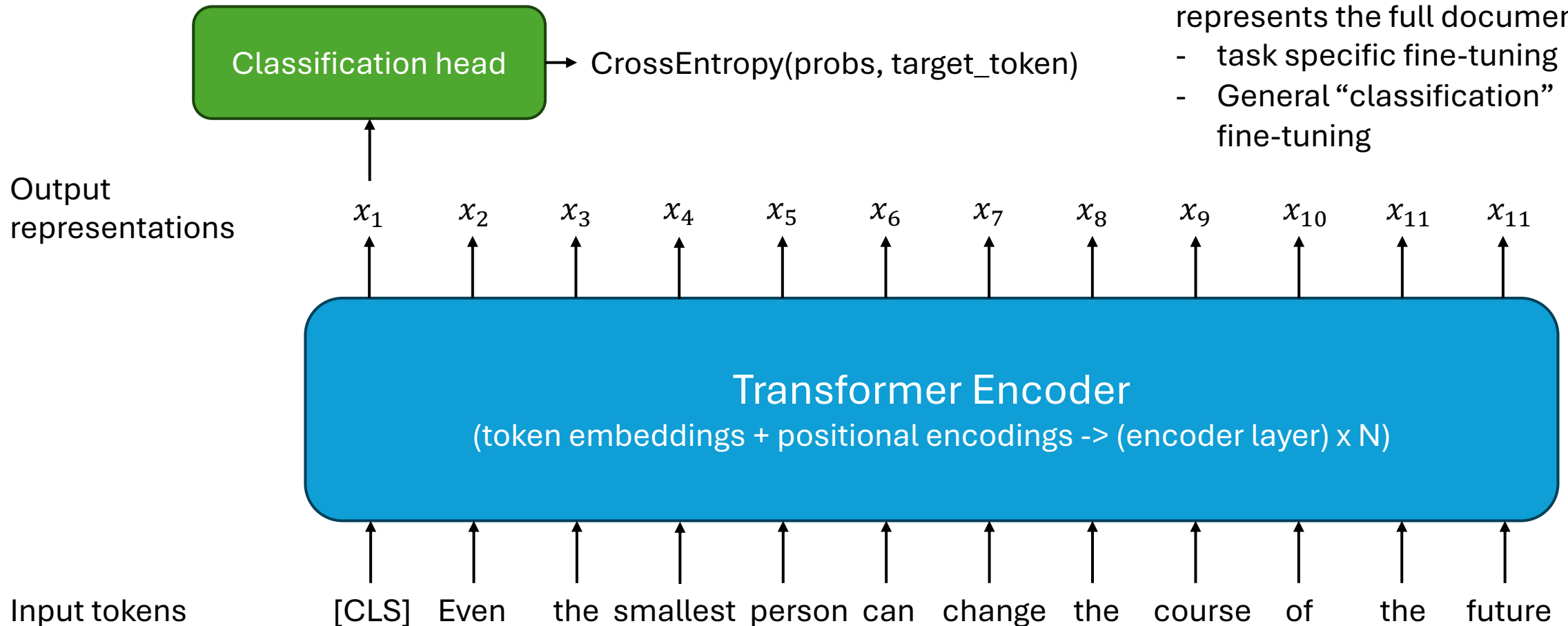


Options for sentence-level prediction

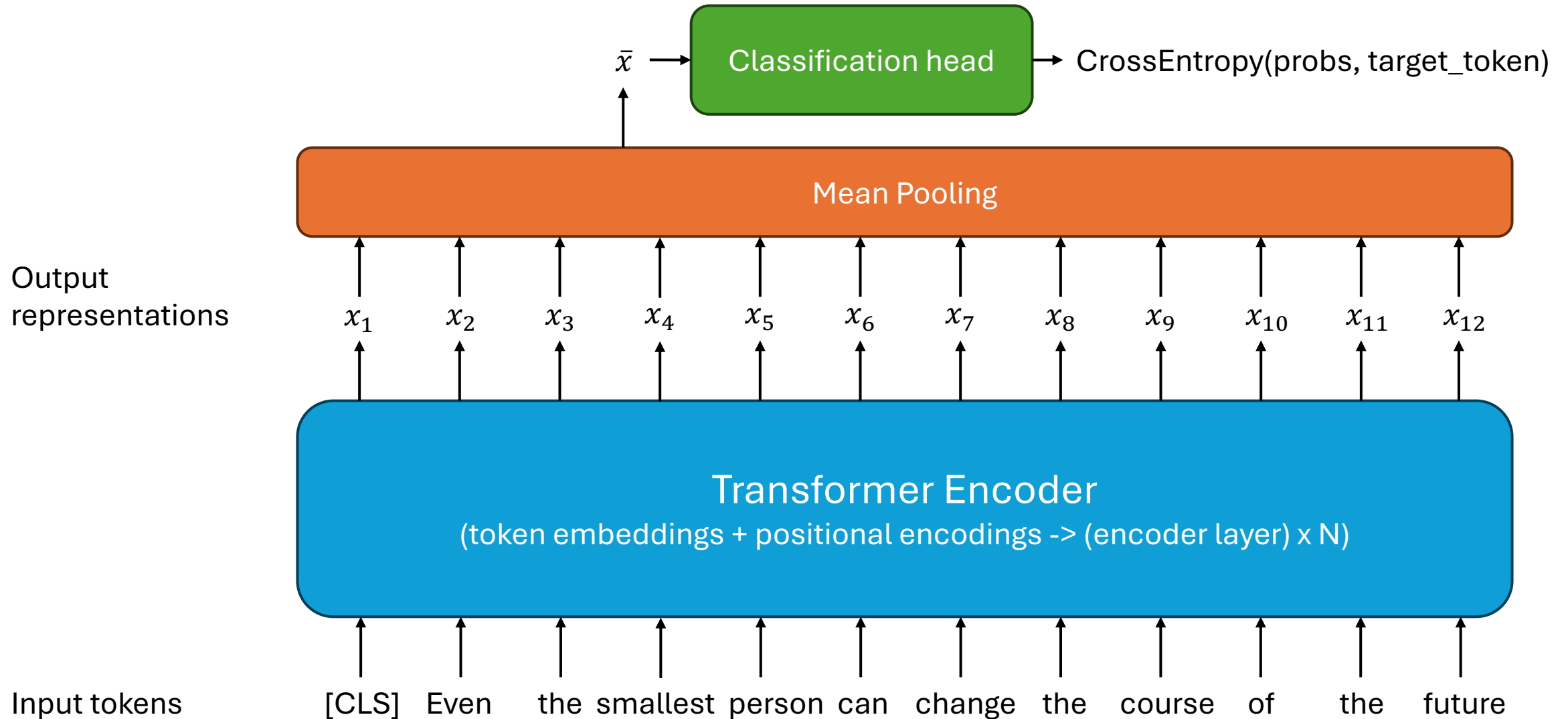
Most common approach: predict from [CLS] token representation

Caveat: requires a tuning strategy to ensure it represents the full document

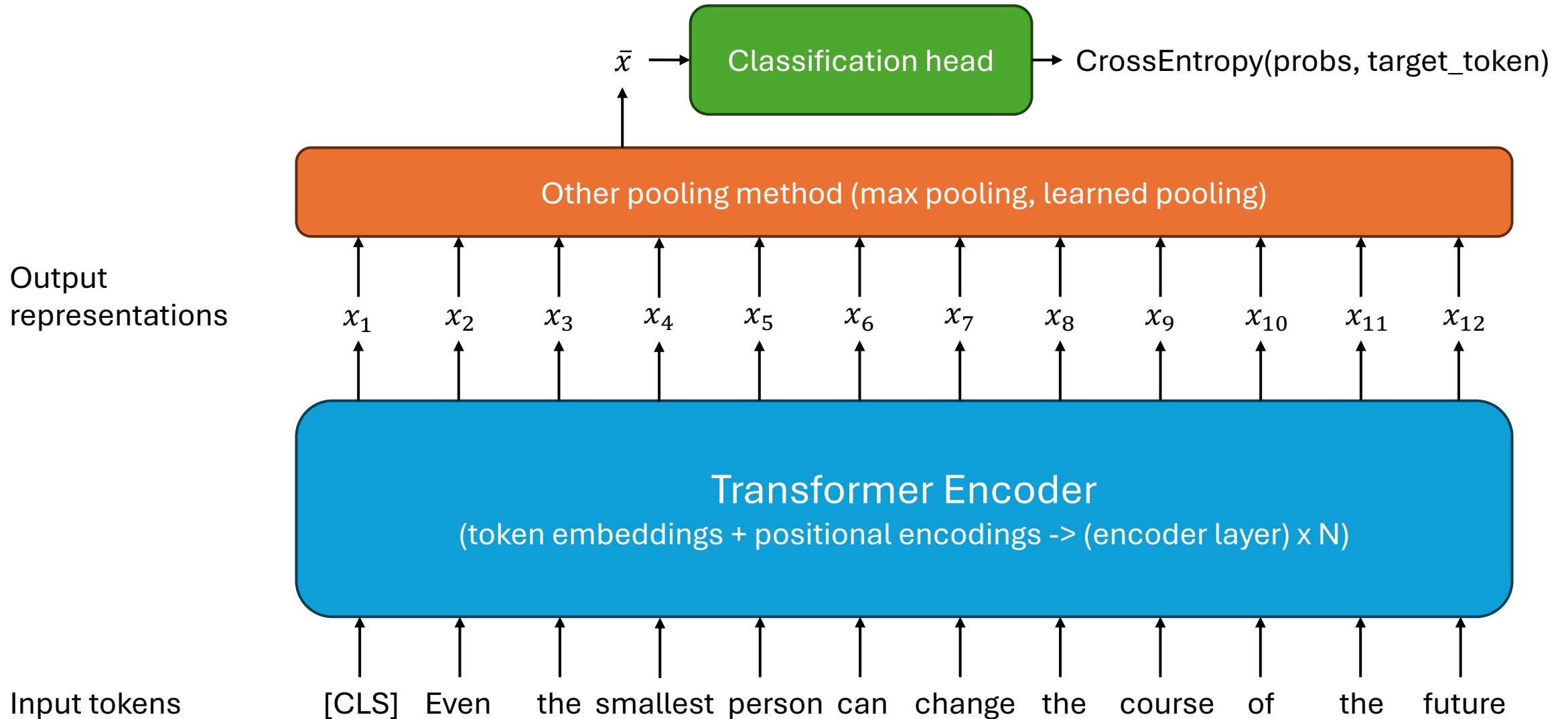
- task specific fine-tuning
- General “classification” fine-tuning



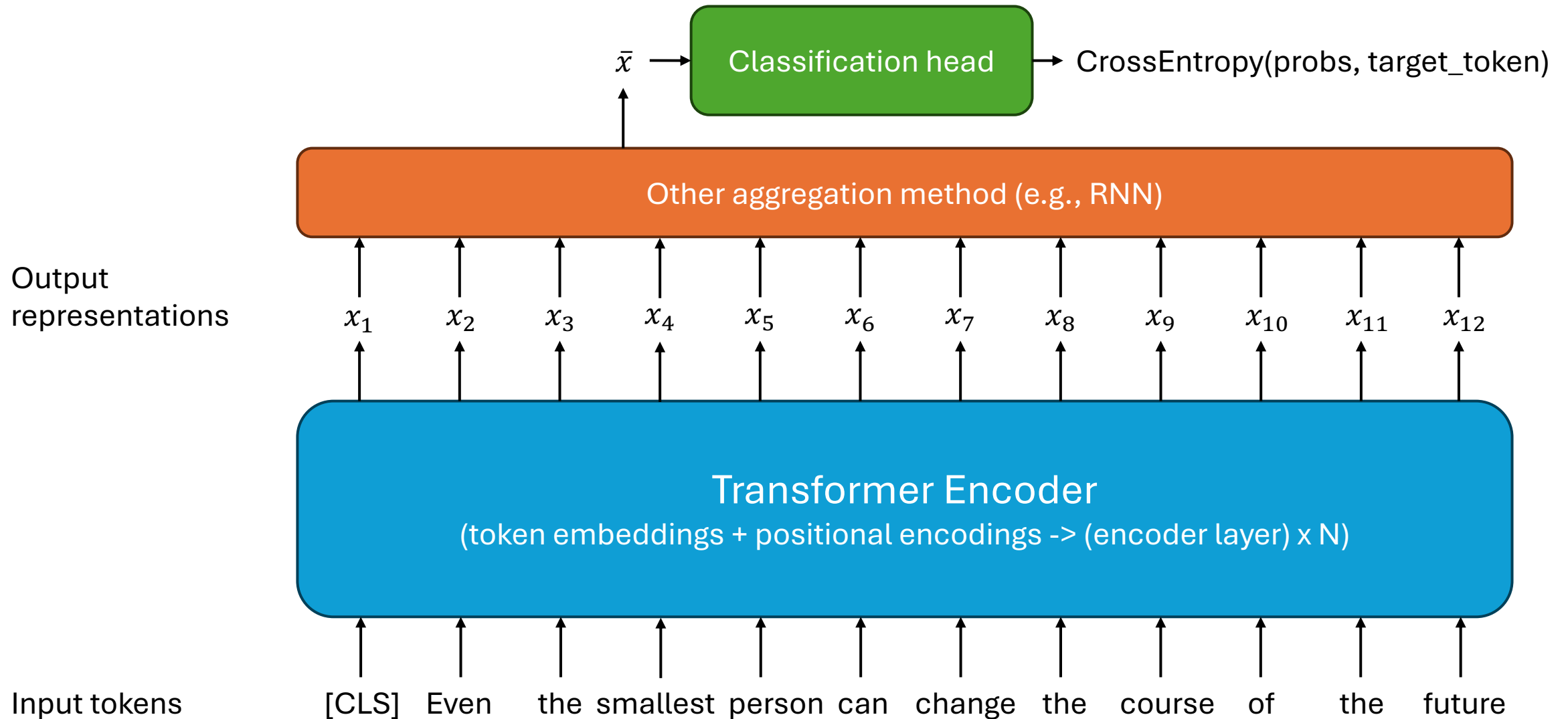
Options for sentence-level prediction



Options for sentence-level prediction



Options for sentence-level prediction



Massive Text Embedding Benchmark (MTEB) leaderboard

Download models tuned for specific types of tasks:

- Classification
- Retrieval
- more

The *zero-shot* column is an indicator of leakage of evaluation tasks into training data

Part of Coding Assignment 3

Spaces

mteb/leaderboard

like 7.23k

Running on CPU UPGRADE

App

Files

Community 182

> Embedding Leaderboard

This leaderboard compares 100+ text and image embedding models across 1000+ languages. We refer to the publication of each selectable benchmark for details on metrics, languages, tasks, and task types. Anyone is welcome [to add a model](#), [add benchmarks](#), [help us improve zero-shot annotations](#) or [propose other changes to the leaderboard](#).

MTEB(eng, v2)

The new English Massive Text Embedding Benchmark. This benchmark was created to account for the fact that many models have now been finetuned to tasks in the original MTEB, and contains tasks that are not as frequently used for model training. This way the new benchmark and leaderboard can give our users a more realistic expectation of models' generalization performance.

The original MTEB leaderboard is available under the [MTEB\(eng, v1\)](#) tab.

Number of languages: 1

Number of tasks: 41

Number of task types: 1

Number of domains: 13

Summary

Performance per Model Size

Performance over Time

Performance per Task Type

Performance per task

Task information

Cite and share this benchmark

Customize this Benchmark

Advanced Model Filters

Rank (Bord...	Model	Zero-shot	Active Parame...	Total Paramet...	Embedding Dim...	Max Tokens	Mean (Ta...	Mean (TaskTyp...	Classification
1	voyage-3-m-exp	0%		6.9	2048	32000	95.49	95.49	95.49
2	Seed1.6-embedding	25%			2048	32768	92.42	92.42	92.42
3	F2LLM-v2-14B	0%	13.2	14.0	5120	40960	91.68	91.68	91.68
4	F2LLM-4B	0%	3.6	4.0	2560	8192	91.68	91.68	91.68
5	F2LLM-v2-4B	0%	3.6	4.0	2560	40960	91.68	91.68	91.68
6	F2LLM-v2-8B	0%	6.9	7.6	4096	40960	91.52	91.52	91.52
7	F2LLM-v2-1.7B	0%	1.4	1.7	2048	40960	91.39	91.39	91.39
8	F2LLM-1.7B	0%	1.4	1.7	2048	8192	90.86	90.86	90.86

Training process

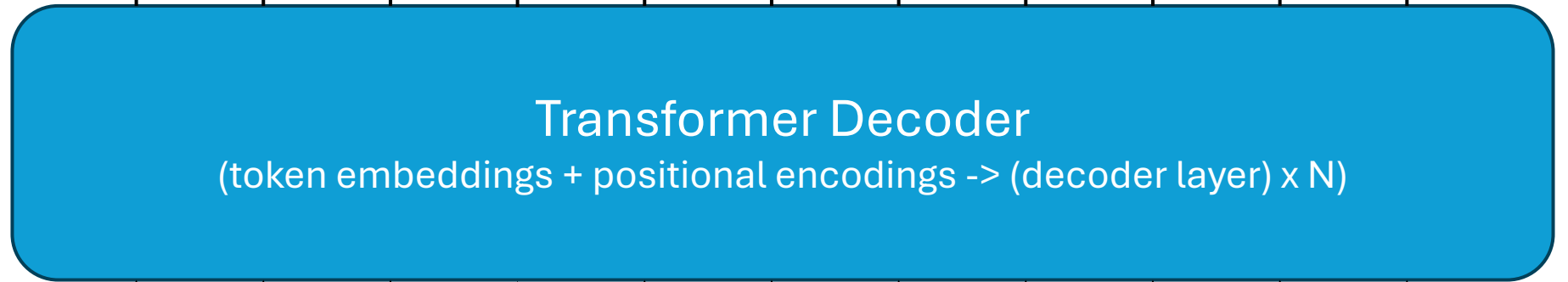
Decoder-only models

Training decoder-only architectures:

Next token prediction (NTP)

Output
representations

x_1 x_2 x_3 x_4 x_5 x_6 x_7 x_8 x_9 x_{10} x_{11}



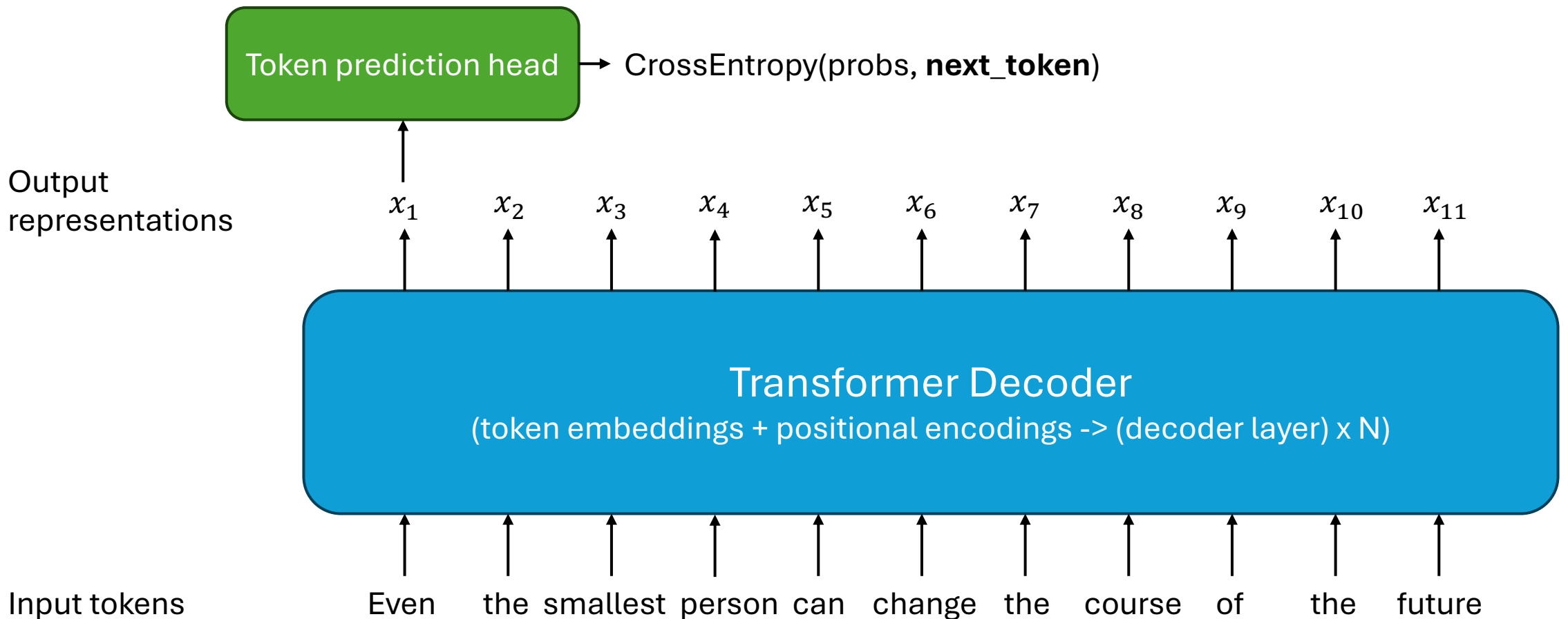
Transformer Decoder
(token embeddings + positional encodings -> (decoder layer) x N)

Input tokens

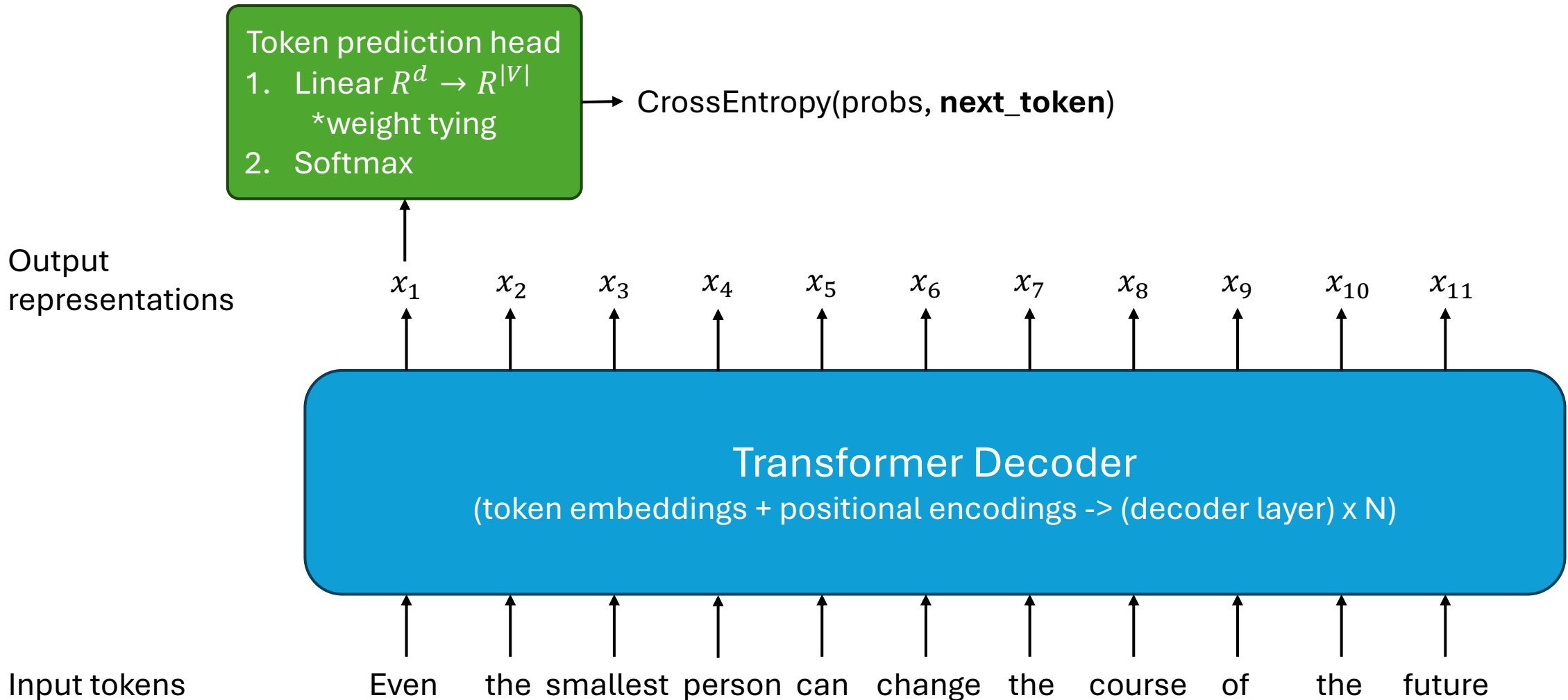
Even the smallest person can change the course of the future

Training decoder-only architectures:

Next token prediction (NTP)

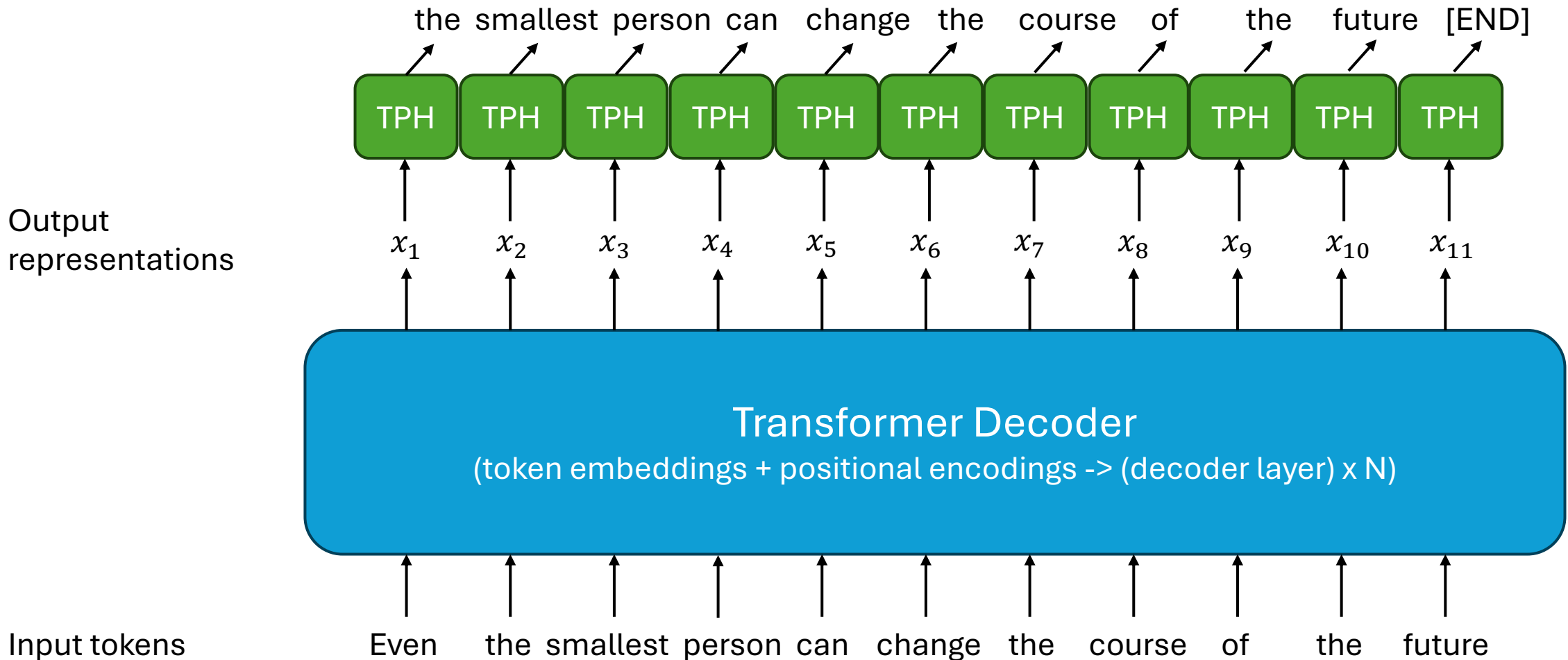


Training decoder-only architectures: Next token prediction (NTP)



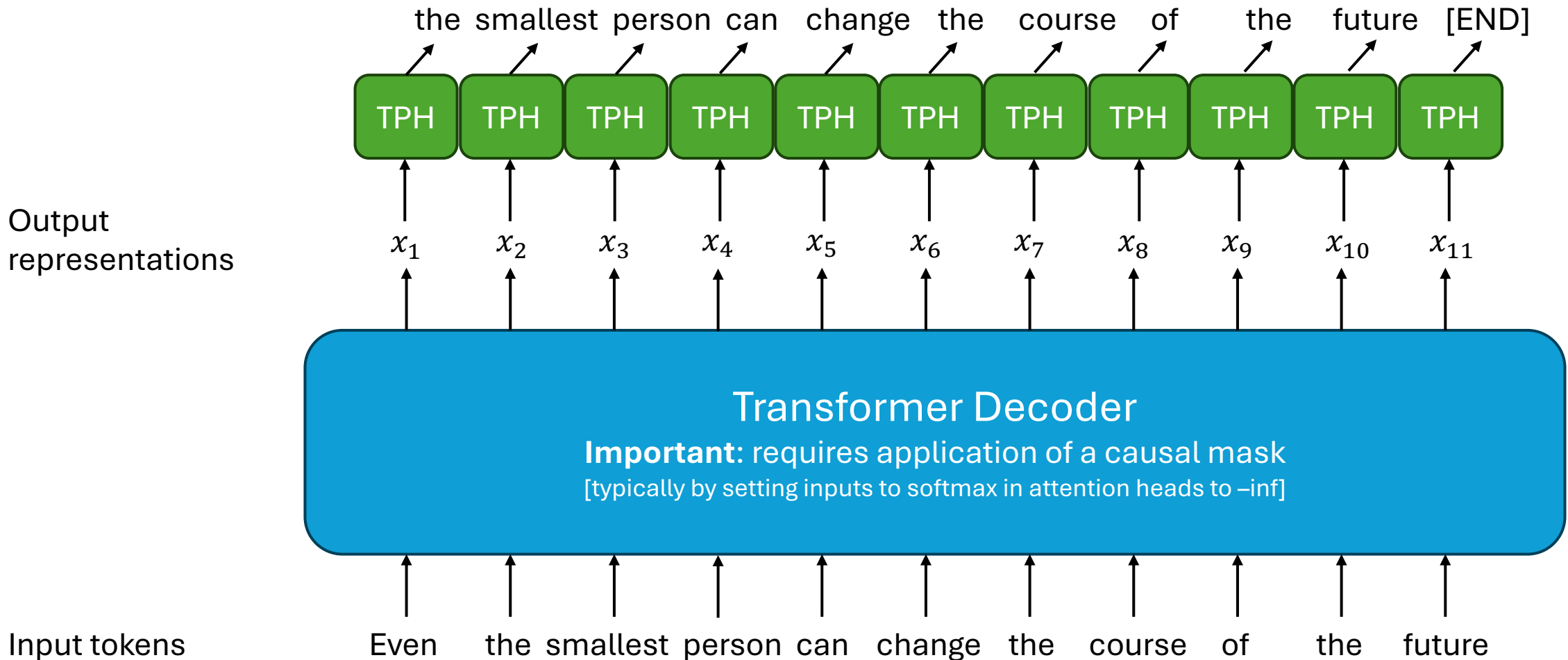
Training decoder-only architectures:

Next token prediction (NTP)



Training decoder-only architectures:

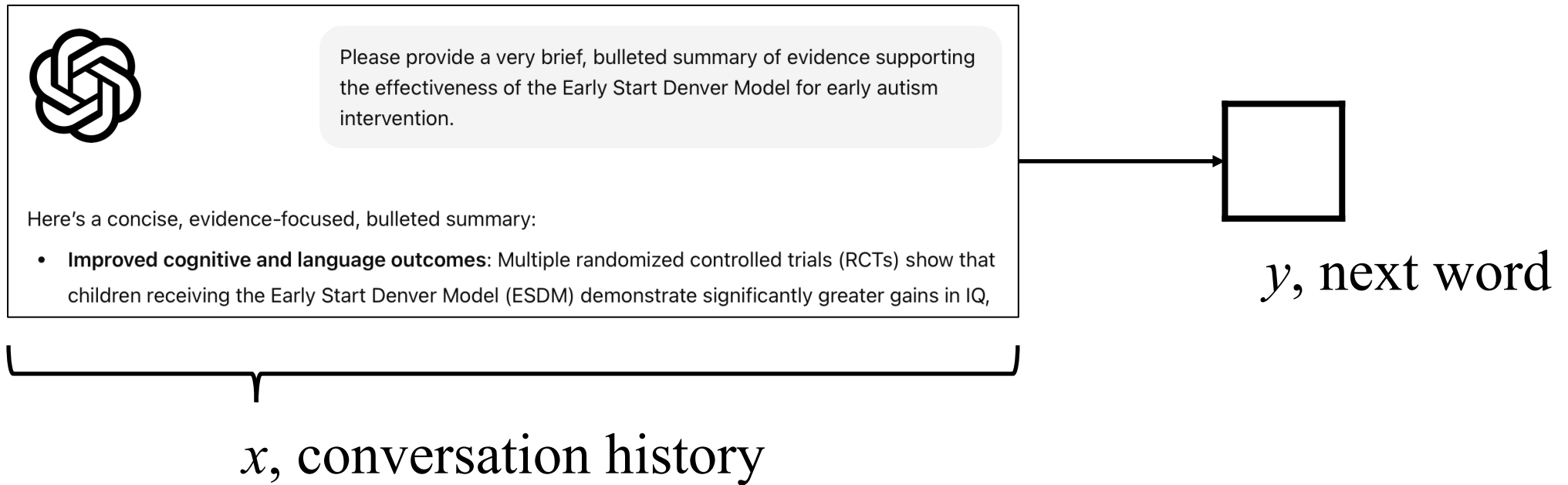
Next token prediction (NTP)



How do we use the output?

Decoder-only models

Now we can generate text!



“next-token prediction”

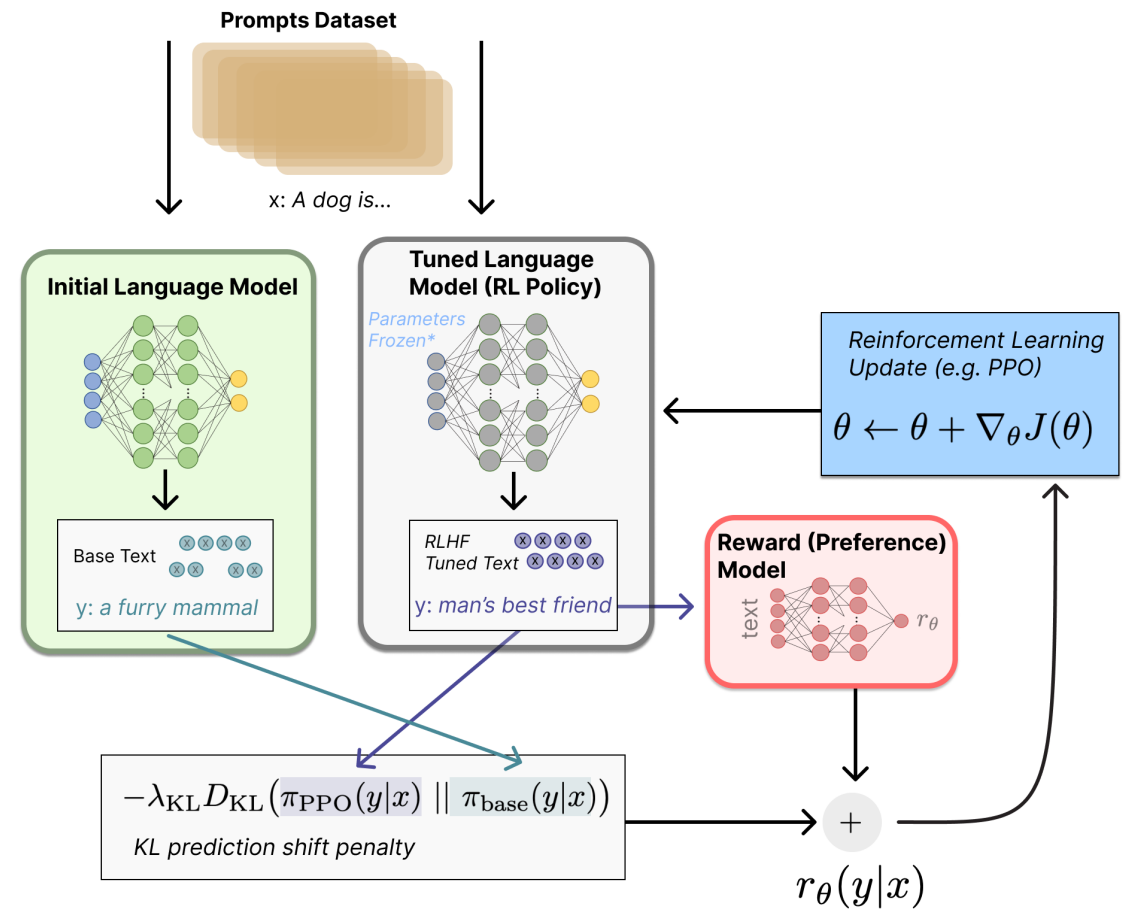
Given a pre-trained model, we can...

- Perform supervised fine-tuning (SFT) on a particular corpus
 - Uses the usual next-token prediction objective
- Align the model with preferences (RLHF) [next slide]
- Additional emerging fine-tuning paradigms
 - e.g. fine-tune to optimize performance on a given rubric
- This is just a preview and mostly beyond our scope

They're then fine-tuned to match our preferences

Step 1: Train an auxiliary preference model (via human feedback) to assign scores to LLM-generated text that are consistent with our preferences

Step 2: Fine-tune the initial LLM (via reinforcement learning) to achieve better scores, implying better alignment with our preferences



Illustrating Reinforcement Learning from Human Feedback (RLHF)
<https://huggingface.co/blog/rlhf>

Case study: maternal health SMS

Case Study: Triaging incoming SMS messages



<https://www.praekelt.org>

How can we use an LLM to solve this problem?

Case Study: Finding instances of mistreatment



<https://www.praekelt.org>

How can we use an LLM to solve this problem?

Notes on the “context window”

- We are often limited by the “context window” of a given model, defined as the maximum number of tokens it can process in a single forward pass
- Sometimes documents are too long to fit
- There are models with extremely long context windows, but they often have difficulty retaining information in the middle of a given document
- Active area of research

Moving to the EHR

A foundation model for EHR data

Models for structured EHR data: the “language” of the EHR

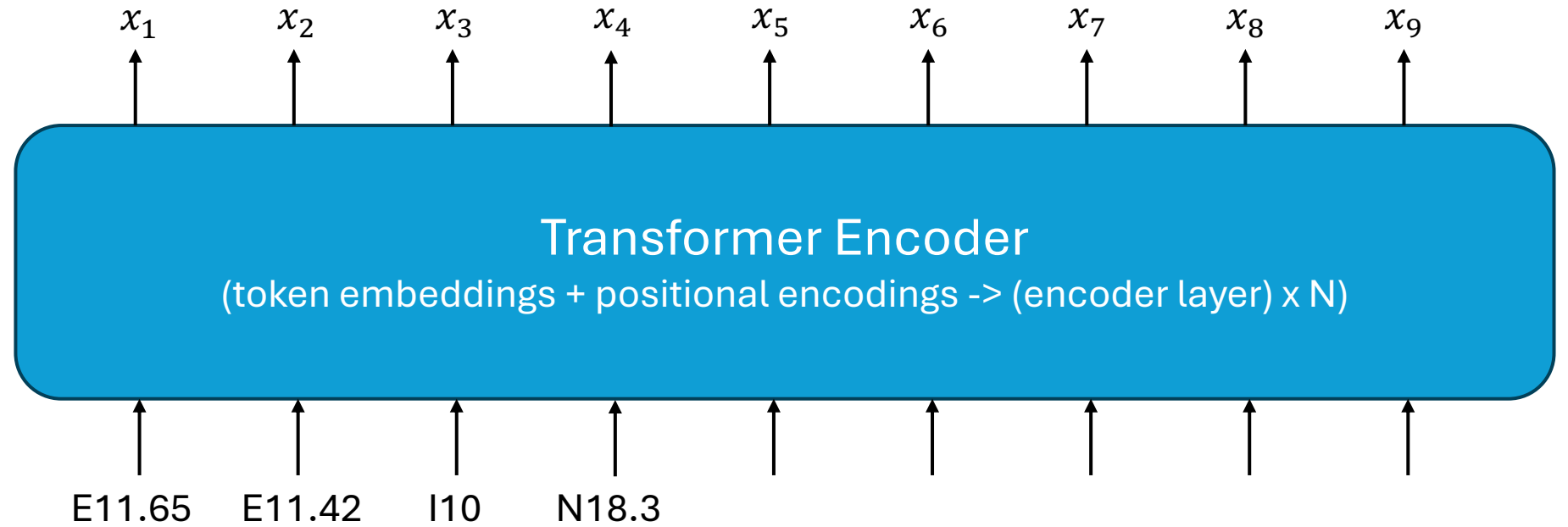
A story about the progression of diabetes...

(“E11.65”, “E11.42”, “I10”, “N18.3”)

- **E11.65** — Type 2 diabetes mellitus with hyperglycemia
- **E11.42** — Type 2 diabetes mellitus with diabetic polyneuropathy
- **I10** — Essential (primary) hypertension
- **N18.3** — Chronic kidney disease, stage 3

Transformers for Electronic Health Records

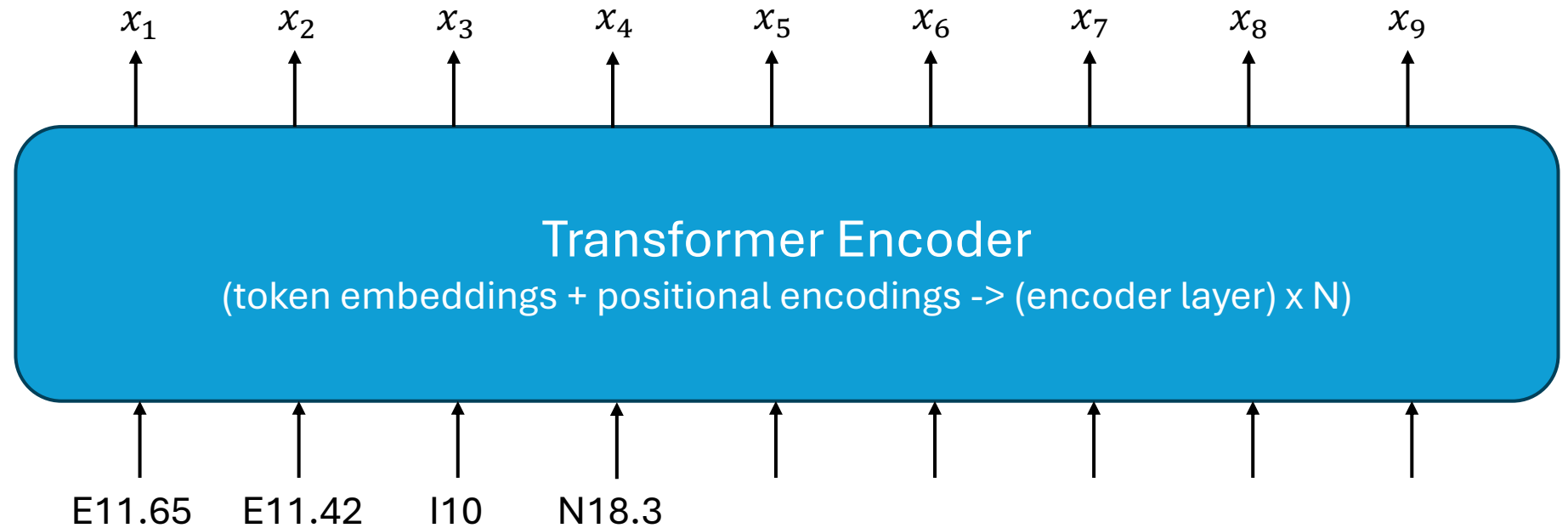
Output
representations



Transformers for Electronic Health Records

Unique consideration 1: we have **position**, **age**, and **time**

Output
representations

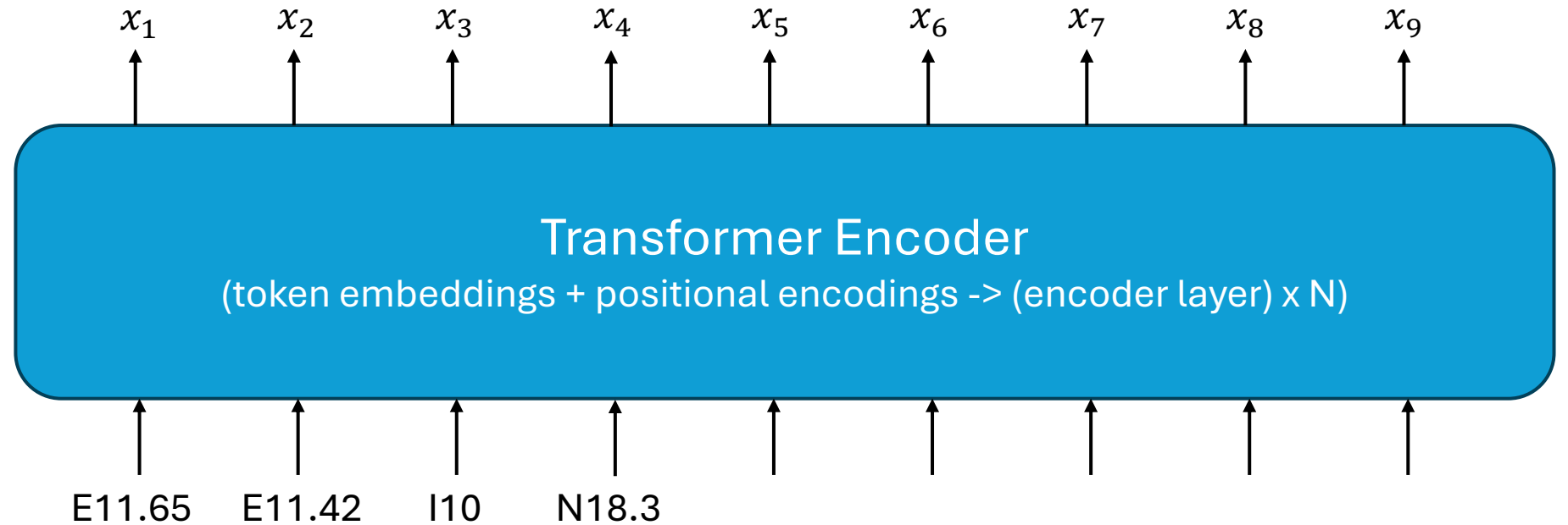


Input tokens

Transformers for Electronic Health Records

Unique consideration 2: some tokens have associated values (e.g., labs, vitals)

Output
representations

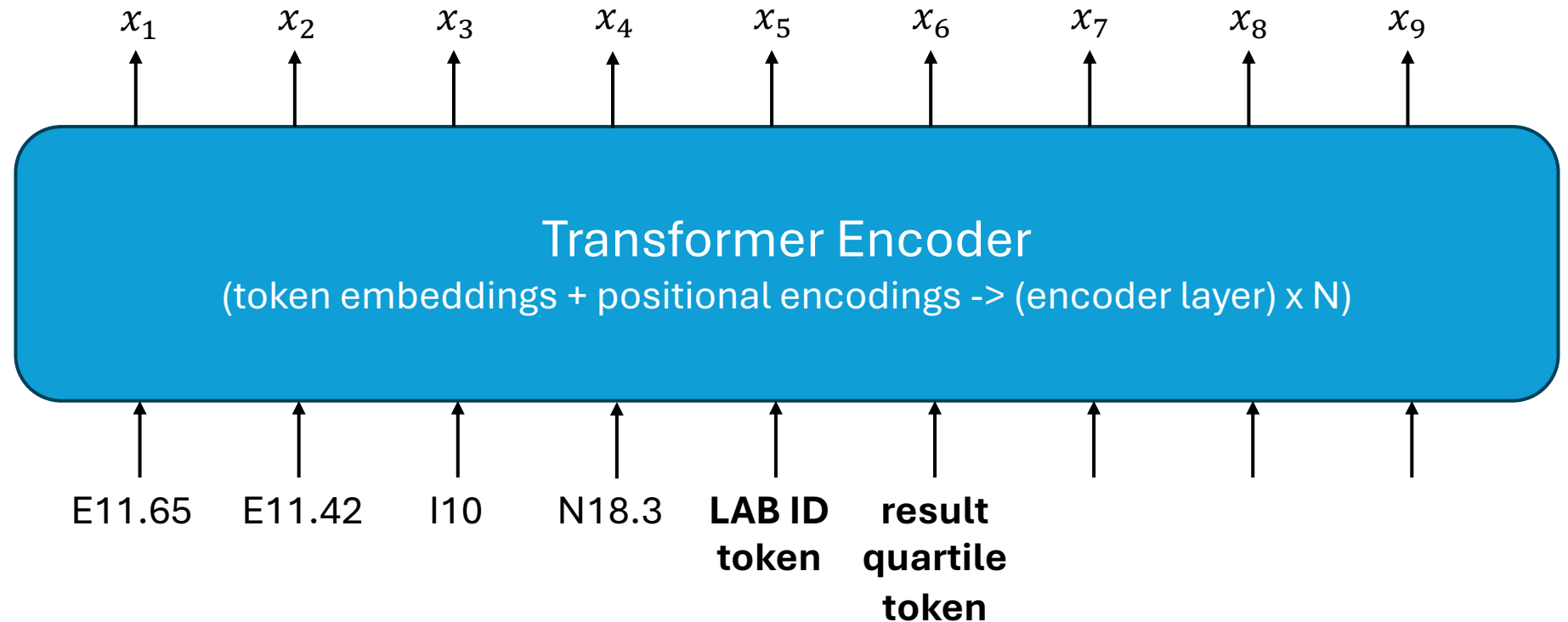


Transformers for Electronic Health Records

Unique consideration 2: some tokens have associated values (e.g., labs, vitals)

One approach to this is shown below

Output
representations



Predict diagnosis risk from structured EHRs

- Suppose we wish to predict which children are most likely to be diagnosed with asthma based on EHRs from early life
- How would we do this without a transformer-based foundation model?
- How would we do this with a transformer-based foundation model?
- What are the advantages of the latter approach?

Some current directions

(optional content)

Recall: are all tasks special cases of text generation?

ME



Please read the following echocardiogram report again and tell me whether left ventricular hypertrophy was detected. This time, please give me your confidence level as a percentage ranging from 0% (i.e. you are certain that it was NOT detected) to 100% (i.e. you are certain that it WAS detected). Answer with the percentage only.

Echo reading:

MILD LV SYSTOLIC DYSFUNCTION (See above) WITH MILD LVH
NORMAL RIGHT VENTRICULAR SYSTOLIC FUNCTION MODERATE
VALVULAR REGURGITATION (See above) NO VALVULAR STENOSIS
NO PERICARDIAL EFFUSION SEVERE BIATRIAL ENLARGEMENT AT
LEAST MILD PULMONARY HTN ELEVATED RA PRESSURES NO
PRIOR STUDY FOR COMPARISON



100%



Generative LLMs enable “zero-shot” prediction:

i.e. prediction without task-specific training

ME



Please read the following echocardiogram report again and tell me whether left ventricular hypertrophy was detected. This time, please give me your confidence level as a percentage ranging from 0% (i.e. you are certain that it was NOT detected) to 100% (i.e. you are certain that it WAS detected). Answer with the percentage only.

Echo reading:

MILD LV SYSTOLIC DYSFUNCTION (See above) WITH MILD LVH
NORMAL RIGHT VENTRICULAR SYSTOLIC FUNCTION MODERATE
VALVULAR REGURGITATION (See above) NO VALVULAR STENOSIS
NO PERICARDIAL EFFUSION SEVERE BIATRIAL ENLARGEMENT AT
LEAST MILD PULMONARY HTN ELEVATED RA PRESSURES NO
PRIOR STUDY FOR COMPARISON



100%



Do we still need predictive AI?

and task-specific models?

Often predictive and generative models are complementary.

Maternal health response system:

- Speech to text (predictive)
- Translation (predictive)
- Identification of key concepts and topics (predictive)
- Triage (predictive)
- Generation of template and/or complete responses in specific cases (generative)

It is much easier to evaluate the performance of predictive models, and in turn to know when and how much to trust them.



In some cases, however, generative models have superseded predictive models.

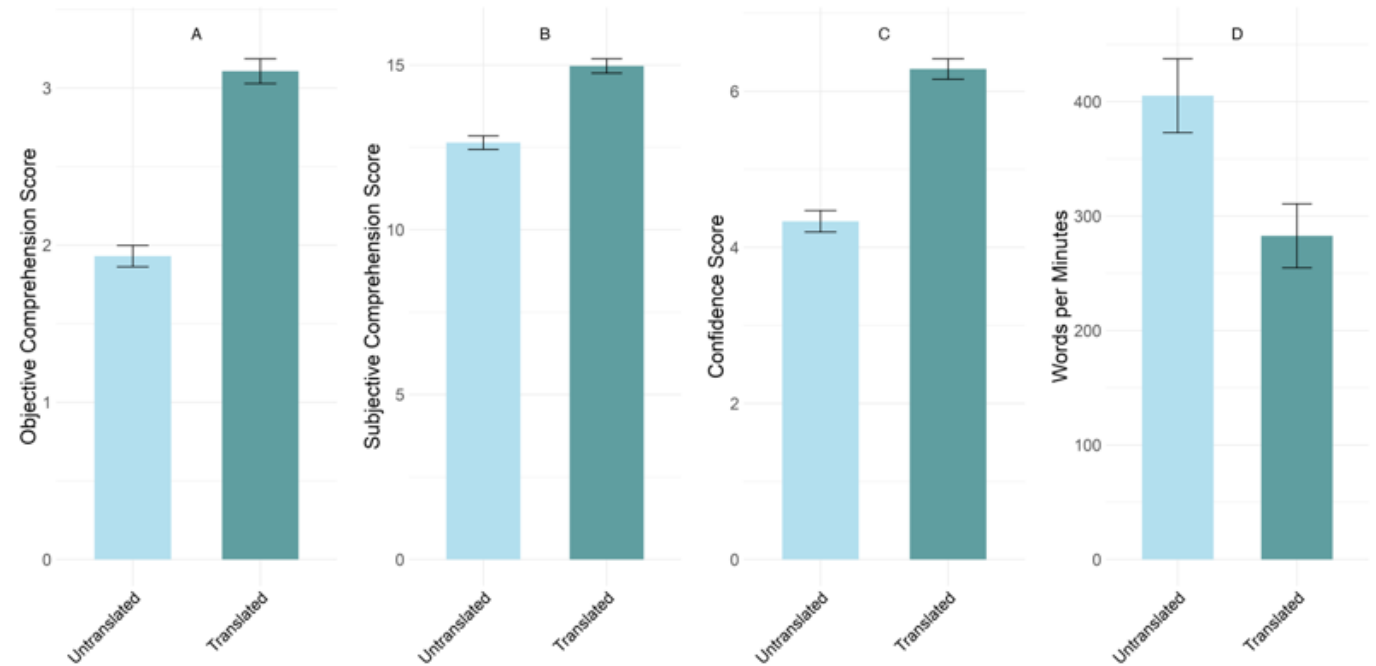
“Translation” of discharge notes into plain language using GPT4 substantially improves patients’ ability to comprehend them.



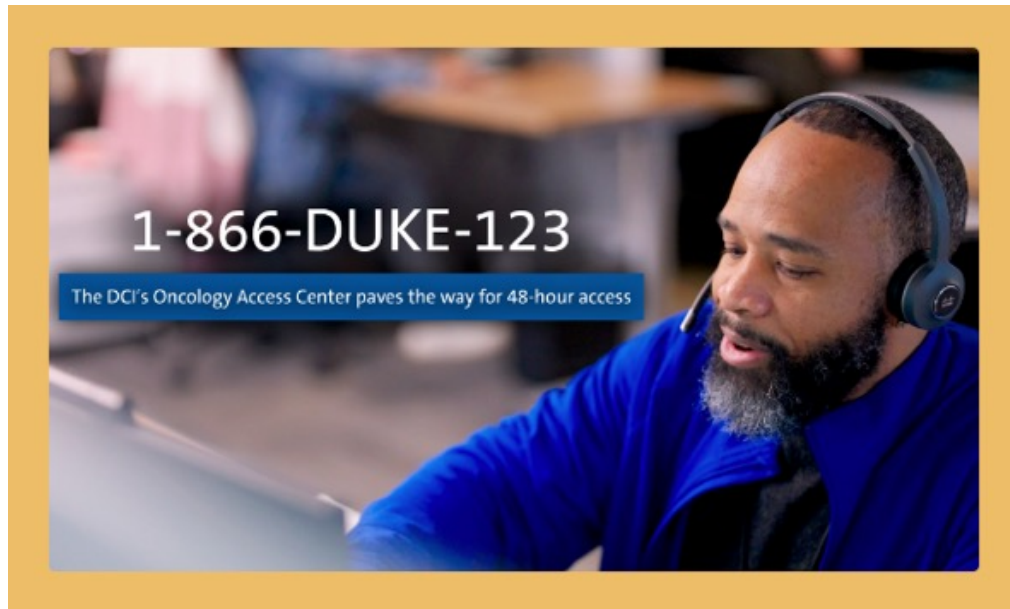
Anivarya Kumar



Isabella Wang



Extracting specific info from documents: predictive or generative?



Data needed to determine scheduling needs
ICD-10
Stage
Age
Date of Dx
Hormone Receptors and molecular biomarkers
History of Treatment (med, surg, rad)
Undiagnosed patients only:
Review of symptoms

While a predictive approach might be more trustworthy in principle, generative models can perform many tasks “zero-shot”, i.e. without task-specific training.



DIHI '24, led by Emily Norboge

Current directions: RAG vs long context

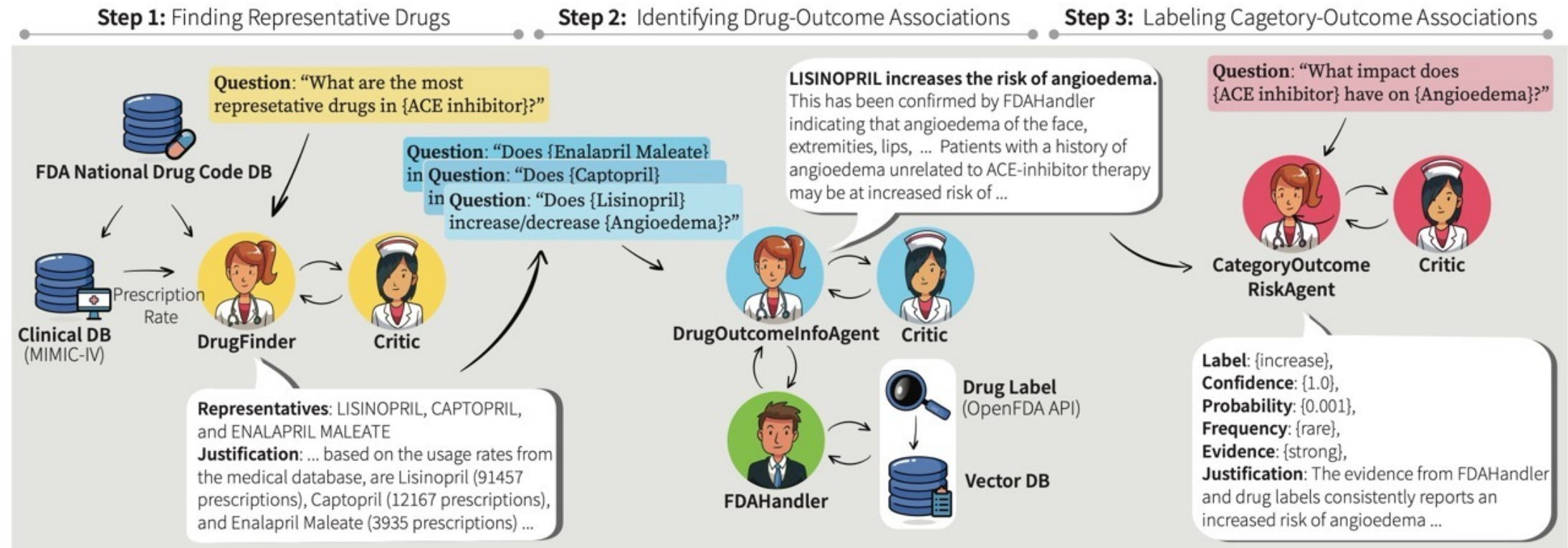
Suppose we wish to answer questions or extract information from a large collection of documents (e.g., all clinical notes for a given patient)

How do we use an LLM to do this?

- Option 1: *very* long context window
- Option 2: retrieval augmented generation (RAG)

Currently, RAG is commonly used because it is effective and scalable, and it (partly) addresses LLM pitfalls, including hallucinations.

Current directions: agentic AI



Multi-LLM-agent systems including:

- An orchestrator agent
- Task-specific agents (e.g. DocChat, SQLchat)
- Critic agents

Current directions: evaluation of generative models

Open questions include:

- Can we trust models to evaluate other models?
- How can models *learn when to defer* to human experts?
- How can we identify LLM biases? (more on this in PIONEER sessions)



Presented by:
Chuan Hong, PhD; Assistant Professor of
Biostatistics & Bioinformatics, Duke
University School of Medicine



AI Health Virtual Seminar Series: Evaluating Generative Large Language Models in Healthcare

Tuesday, April 16, 2024 | 12:00 PM – 1:00 PM (Eastern time)
Virtual seminar via Zoom, open to members internal and external to Duke

Register here: https://duke.zoom.us/webinar/register/WN_4eaOxm6KRXahb8p3H0d3nQ

The rapid evolution of large language models (LLMs) has ushered in a new era of computational linguistics, yet a systematic approach to their evaluation, particularly in sensitive domains such as healthcare, remains nascent. This work bridges these gaps by offering a detailed and integrated review of qualitative evaluation, quantitative evaluation, and meta-evaluation. For quantitative evaluation, our review introduces a taxonomy of evaluation metrics, categorizing them based on essential dimensions such as human supervision, contextual data, and analytical depth. In addition to generic settings, our work distinctively emphasizes additional considerations vital in the healthcare sector. As a result, we propose an integrated cross-walk between qualitative and quantitative assessment methods. The proposed framework harmonizes qualitative insights, such as user-focused evaluations, with objective quantitative metrics. We present a detailed "go-to menu" of evaluation criteria, tailored to address specific healthcare applications and emphasize distinct aspects in both pre-deployment and post-deployment phases. Our findings underscore the need for evaluations that extend beyond mere technical accuracy, factoring in medical ethics, fairness, equity, and potential operational biases. Our work offers a summary of existing methods of LLM evaluation that can establish a baseline from which future evaluation methods can be developed to keep pace with the rapid advancements in the field.

